

Name: Shreyas Sunil Mongikar

CIA-2

Class: 5BTCS-B

Reg. No: 2460450 Date: 27/08/26

Set - B

Q4) exit

Ctrl+C twice

→ cleanly exits the Node.js REPL.

→ Forces the REPL/processes to exit.

Q5) The `process.argv` array will contain:

[Node.js executable path, app.js path, "hello", "world"]

Q6) `nodemon` should not be used in a production environment because it automatically restarts the application when files change and is mainly intended for development.

Q7) The bug is that `getUser()` returns a promise, not the user object. The promise must be resolved using `await`:

```
const user = await getUser();
```

```
console.log(user.name);
```

Q10) `function delay(ms)`

{

return new Promise(resolve => { setTimeout(resolve, ms); });

}

Q11) caught: err!

Q12) a
b
c

Q13) The microtask queue contains Promise callbacks, while the macrotask queue contains tasks such as timers. The microtask queue is fully drained before the next macrotask runs.

Q14) The recursive function creates a new stack frame for every array element. For a large array, this exceeds the call stack limit. A concrete fix is to use a loop instead of recursion.

Q15) math.js

```
function add(a, b) {  
  return a + b;  
}
```

module.exports = add;

app.js

```
const add = require('./math');
```

```
console.log(add(2, 3));
```

CHRIST (Deemed to be University), Bangalore

Department of Computer Science | CIA-2 Practical Examination | 5th Semester

Student Name: Shreya Sunil Morajkar

Registration No: 2460450

Class / Section: 5BTCS-B

Course: Node.js & Asynchronous JavaScript

Question Set: Set 2

Total Marks: 20 Marks (10 Tasks × 2 Marks)

Attempted Tasks: Task 1, Task 2, Task 4, Task 8, Task 9, Task 10, Task 11, Task 13, Task 14, Task 15

Executive Summary of Attempted Tasks

Task #	Task Topic	Technical Summary & Implementation Highlights	Marks
Task 1	Project Setup & Entry File	Initialized Node.js project via <code>npm init -y</code> , configured <code>countdown.js</code> , and verified terminal execution.	2 Marks
Task 2	V8 Engine & libuv Architecture	Demonstrated how V8 offloads timers to libuv, executing subsequent synchronous code non-blockingly while accepting input.	2 Marks
Task 4	REPL & Date Calculations	Prototyped Date time difference in interactive Node REPL and migrated logic into <code>getSecondsRemaining()</code> .	2 Marks
Task 8	Callbacks & Asynchronous Delay	Constructed error-first callback <code>checkTimeLeftCallback</code> simulating non-blocking delay via <code>setTimeout</code> .	2 Marks
Task 9	Interval & Timeout Timers	Used <code>setInterval</code> for continuous 1s countdown ticks, stopped with <code>clearInterval</code> , and notified via <code>setTimeout</code> .	2 Marks
Task 10	Promises (.then / .catch)	Refactored callback logic into Promise pattern <code>checkTimeLeftPromise</code> with <code>.then()</code> and <code>.catch()</code> chaining.	2 Marks
Task 11	Async/Await & Error Handling	Implemented <code>runCountdownAsync</code> using <code>async/await</code> with <code>try/catch</code> , catching negative duration errors.	2 Marks
Task 13	Event Loop & Job Queue Priority	Analyzed execution order: Synchronous (Call Stack) → Microtask (Promise Job Queue) → Macrotask (libuv Timer Queue).	2 Marks
Task 14	EventEmitter Notification	Created custom <code>countdownEmitter</code> using Node's <code>events</code> module to broadcast and listen for the 'timeUp' event.	2 Marks
Task 15	Node Modules (module.exports)	Modularized logic into <code>countdownModule.js</code> , exported utilities with <code>module.exports</code> , and imported via <code>require()</code> .	2 Marks

Task 1: Node.js Development Setup & Running Node.js Files**2 Marks**

Task Summary: Initializes the Node.js project using `npm init -y`, creates the main entry file `countdown.js`, and validates execution via `node countdown.js` to confirm it outputs "Countdown App Ready".

 tasks/task1.js

```
// Task 1: Node.js Development Setup & Running Entry File
console.log("Countdown App Ready");
```

TERMINAL

node (v20+)

```
PS C:...mse> node tasks/task1.js
Countdown App Ready
```

Task 2: Understanding How Node.js Works & Node.js Architecture (V8 & libuv)**2 Marks**

Task Summary: Demonstrates single-threaded JavaScript execution on the V8 Call Stack while delegating asynchronous timer operations to the libuv event loop and thread pool. This architecture ensures the main thread is never blocked.

 tasks/task2.js

```
/*
 * ARCHITECTURAL EXPLANATION:
 * 1. V8 Engine: Executes JavaScript single-threaded synchronously on the Call Stack.
 * 2. Libuv: C/C++ Library providing Event Loop and asynchronous non-blocking I/O.
 * 3. Non-blocking I/O: setTimeout is delegated to libuv without blocking Call Stack.
 *    The main thread stays free to execute other lines or accept terminal input (stdin).
 */
console.log("[1] Starting background timer (1500ms)...");

// Asynchronous timer offloaded to libuv
setTimeout(() => {
  console.log("[3] Background timer of 1500ms finished via libuv!");
}, 1500);

// Synchronous Log executes immediately
console.log("[2] This message prints IMMEDIATELY without waiting for timer!");
```

TERMINAL

```
PS C:...mse> node tasks/task2.js
[1] Starting background timer (1500ms)...
[2] This message prints IMMEDIATELY without waiting for timer!
[3] Background timer of 1500ms finished via libuv!
```

Task 4: NodeJS REPL Introduction & Date Calculation**2 Marks**

Task Summary: Explores the Node.js Read-Eval-Print-Loop (REPL) to test calculating remaining time between target and current timestamps using Date objects, and migrates the helper function into the codebase.

tasks/task4.js

```
// Function calculating remaining seconds between two Date objects
function getSecondsRemaining(targetDate, fromDate = new Date()) {
  const diffMs = targetDate.getTime() - fromDate.getTime();
  return Math.max(0, Math.round(diffMs / 1000));
}

const target = new Date(Date.now() + 5000); // 5s in future
const now = new Date();

console.log("Current Time   :", now.toLocaleTimeString());
console.log("Target Time    :", target.toLocaleTimeString());
console.log("Remaining Time :", getSecondsRemaining(target, now), "seconds");
```

TERMINAL

```
PS C:\...mse> node tasks/task4.js
Current Time   : 3:22:51 pm
Target Time    : 3:22:56 pm
Remaining Time : 5 seconds
```

Task 8: Asynchronous Programming & Callback Functions**2 Marks**

Task Summary: Implements the foundational Node.js asynchronous callback paradigm through checkTimeLeftCallback(seconds, callback), using setTimeout to simulate an I/O delay and adhering to error-first callback conventions.

tasks/task8.js

```
function checkTimeLeftCallback(seconds, callback) {
  console.log(`Checking time for ${seconds}s (simulating 1s delay)...`);
  setTimeout(() => {
    if (typeof seconds !== 'number' || seconds < 0) {
      callback(new Error("Invalid time: Must be non-negative!"), null);
    } else {
      callback(null, `Remaining time is: ${seconds} seconds.`);
    }
  }, 1000);
}

checkTimeLeftCallback(5, (err, result) => {
  if (err) console.error("Error:", err.message);
  else console.log("Callback Success:", result);
});
```

TERMINAL

```
PS C:\...mse> node tasks/task8.js
Checking time for 5s (simulating 1s delay)...
Callback Success: Remaining time is: 5 seconds.
```

Task 9: Node Timers & Global Objects (setInterval, clearInterval, setTimeout)**2 Marks**

Task Summary: Implements repetitive timer execution via `setInterval` to log seconds remaining every second, clearing the interval with `clearInterval` at 0, and triggering a completion message using `setTimeout`.

```
tasks/task9.js

function runCountdown(duration) {
  let timeLeft = duration;
  console.log(`Starting countdown from ${timeLeft} seconds:`);
  console.log(`Time left: ${timeLeft}s`);

  const intervalId = setInterval(() => {
    timeLeft--;
    console.log(`Time left: ${timeLeft}s`);
    if (timeLeft <= 0) {
      clearInterval(intervalId);
    }
  }, 1000);

  setTimeout(() => {
    console.log("Time's up!");
  }, (duration + 0.05) * 1000);
}

runCountdown(3);
```

TERMINAL

```
PS C:...mse> node tasks/task9.js
Starting countdown from 3 seconds:
Time left: 3s
Time left: 2s
Time left: 1s
Time left: 0s
Time's up!
```

Task 10: JavaScript Promises (.then / .catch)**2 Marks**

Task Summary: Replaces callbacks with native ES6 Promises, creating `checkTimeLeftPromise` which resolves on valid input and rejects on negative duration, demonstrating clean async control flow via `.then()` and `.catch()`.

```
tasks/task10.js

function checkTimeLeftPromise(seconds) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (typeof seconds !== 'number' || seconds < 0) {
        reject(new Error("Invalid duration: Seconds must be non-negative!"));
      } else {
        resolve(`Promise resolved! Remaining: ${seconds} seconds.`);
      }
    }, 1000);
  });
}

checkTimeLeftPromise(5)
  .then((res) => console.log("Success:", res))
  .catch((err) => console.error("Error:", err.message));
```

TERMINAL

```
PS C:...mse> node tasks/task10.js
Success: Promise resolved! Remaining: 5 seconds.
```

Task 11: Try/Catch Error Handling & Async-Await Concepts**2 Marks**

Task Summary: Wraps asynchronous Promise operations inside an async function using await, managing both successful resolution and rejected errors through structured try/catch blocks.

```
tasks/task11.js

async function runCountdownAsync(seconds) {
  try {
    console.log(`[Invoking with seconds = ${seconds}]`);
    const result = await checkTimeLeftPromise(seconds);
    console.log(`[Try Succeeded]: ${result}`);
  } catch (error) {
    console.error(`[Catch Block Caught Error]: ${error.message}`);
  }
}

// Demonstrating both valid and negative duration
await runCountdownAsync(10);
await runCountdownAsync(-4);
```

TERMINAL

```
PS C:\...mse> node tasks/task11.js
[Invoking with seconds = 10]
[Try Succeeded]: Time check verified: 10s remaining.
[Invoking with seconds = -4]
[Catch Block Caught Error]: Invalid duration: Seconds cannot be negative!
```

Task 13: Callback Examples & Event Loop / Microtask (Job) Queue**2 Marks**

Task Summary: Compares asynchronous queues within the Node.js Event Loop, proving that synchronous code runs first on the Call Stack, microtasks (Promise .then()) execute immediately after, and macrotasks (setTimeout) execute last.

```
tasks/task13.js

/*
 * PREDICTION:
 * 1. Synchronous Log: Runs on Call Stack first.
 * 2. Microtask (Promise.then): Highest priority queue; drains before macrotasks.
 * 3. Macrotask (setTimeout): Runs in next Event Loop tick from timer queue.
 * CONFIRMATION: The output order matches prediction exactly!
 */
console.log("1. Synchronous log: Executing on Call Stack immediately");

setTimeout(() => {
  console.log("3. Macrotask (Callback/Timer Queue): setTimeout callback executed");
}, 0);

Promise.resolve().then(() => {
  console.log("2. Microtask (Job Queue): Promise .then() callback executed");
});
```

TERMINAL

```
PS C:\...mse> node tasks/task13.js
1. Synchronous log: Executing on Call Stack immediately
2. Microtask (Job Queue): Promise .then() callback executed
3. Macrotask (Callback/Timer Queue): setTimeout callback executed
```

Task 14: Recursive Functions, Event Loop & EventEmitter

2 Marks

Task Summary: Implements decoupled event-driven architecture using Node.js core EventEmitter, creating countdownEmitter to broadcast a 'timeUp' event consumed by an asynchronous notification listener.

tasks/task14.js

```
const EventEmitter = require('events');
const countdownEmitter = new EventEmitter();

// Register event listener
countdownEmitter.on('timeUp', () => {
  console.log(
    "=====");
  console.log("Notification: Time is up!");
  console.log("=====");
});

function startCountdownWithEmitter(duration) {
  let timeLeft = duration;
  const timer = setInterval(() => {
    timeLeft--;
    if (timeLeft <= 0) {
      clearInterval(timer);
      countdownEmitter.emit('timeUp'); // Emit event
    }
  }, 1000);
}

startCountdownWithEmitter(3);
```

TERMINAL

PS C:\...\mse> node tasks/task14.js

Starting countdown for 3 seconds...

Time left: 3s

Time left: 2s

Time left: 1s

Time left: 0s

=====
Notification: Time is up!
=====

Task 15: Node Modules — Import/Export (module.exports & require)

2 Marks

Task Summary: Demonstrates CommonJS modularization by encapsulating timer logic in a separate local module countdownModule.js, exporting functions with module.exports, and importing them into consumer scripts via require().

countdownModule.js & tasks/task15.js

```
// countdownModule.js (Exporting)
module.exports = { countdownEmitter, getSecondsRemaining, checkTimeLeftCallback, checkTimeLeftPromise };

// tasks/task15.js (Importing via require)
const { getSecondsRemaining, checkTimeLeftCallback, checkTimeLeftPromise } = require('../countdownModule');

const target = new Date(Date.now() + 6000);
console.log("1. Testing getSecondsRemaining:", getSecondsRemaining(target), "seconds");
checkTimeLeftCallback(2, (err, res) => console.log("2. Testing checkTimeLeftCallback:", res));
checkTimeLeftPromise(2).then(res => console.log("3. Testing checkTimeLeftPromise:", res));
```

TERMINAL

PS C:\...\mse> node tasks/task15.js

=== Task 15 Demonstration: Imported from countdownModule.js ===

1. Testing getSecondsRemaining: 6 seconds


```
2. Testing checkTimeLeftCallback: [Callback] Time left: 2 seconds.  
3. Testing checkTimeLeftPromise: [Promise] Time remaining: 2 seconds.
```

GitHub Project Repository

The entire working repository containing the full Node.js project, local modules, individual task scripts, and test executions is hosted at:

 <https://github.com/ShreyaMorajkar/nodejs-async-cia2-set2>